

SSD_FASTBOOT

usage reference

1. Basic introduction

1.1. Overview

This document will describe the configuration and use of Quick Start.

The main principle of FASTBOOT is to put the necessary modules and user apps related to the touch screen on the ramdisk to start in advance, so as to realize the function of fast bright screen.

- advantage

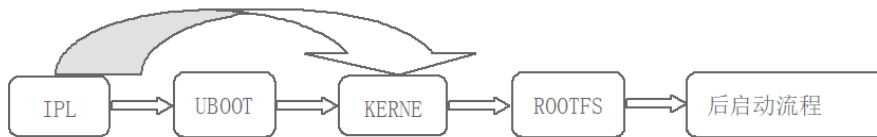
Can achieve fast screen brightening.

- shortcoming

Making a ramdisk requires extra memory. The larger the app size is, the larger the ramdisk will be, and the more memory it will consume. Therefore, before deciding to use fastboot, it is necessary to evaluate whether the memory is sufficient.

Note: If the user app is very large, it is not recommended to use fastboot.

1.2. Start the process



- By default, fastboot will skip uboot and start the kernel directly
 - If you need to enter uboot, you can long press enter, reboot until you enter uboot
 - The post-start process refers to the profile execution stage after the system is up.
-

1.3. Keyword Description

- RAMFS

RAMFS is a ram-based file system under Linux, so ramfs will have higher efficiency.

One flaw of ramfs is that it takes up a lot of memory, because the rootfs is put into the memory area, even if you specify the size when you mount it.

- dlopen

Linux provides a set of APIs to dynamically load libraries. These APIs are listed below:

dlopen to open a library and prepare it for use.

dlsym, to find the value of a symbol in an open library.

dlclose, closes the library.

dLError, returns a string describing the error message of the last call to dlopen, dlsym, or dlclose.

2. Public version related instructions

2.1. Compile configuration

SSD201/SSD202 public version has added the following defconfig.

*The memory required for the Fastboot function is relatively large, because the public version has many behaviors and UIs, and the actual development test performed on 202 (128M memory) is shown in blue in the following table.

The subsequent practical examples are all performed in
configs/nvr/i2m/8.2.1/spinand.ram-glibc-squashfs.011a.128.

Chip	Flash	Toolchain	Other	Defconfig
SSD201	NOR	glibc	NA	configs/nvr/i2m/8.2.1/nor.glibc-ramfs.011a.64
SSD202	NOR	glibc	NA	configs/nvr/i2m/8.2.1/nor.glibc-ramfs.011a.128
SSD201	NAND	glibc	NA	configs/nvr/i2m/8.2.1/spinand.ram-glibc-squashfs.011a.64
SSD202	NOR	glibc	NA	configs/nvr/i2m/8.2.1/spinand.ram-glibc-squashfs.011a.128

2.2. Introduction to the post-start process

In addition to skipping uboot to reduce the boot time, the Fastboot function also has a quick way to display the screen by dividing the original ko initialization place into two parts:

- The necessary parts of the touch screen and UI are placed in the init stage.
- Other parts are still in the original demo stage.

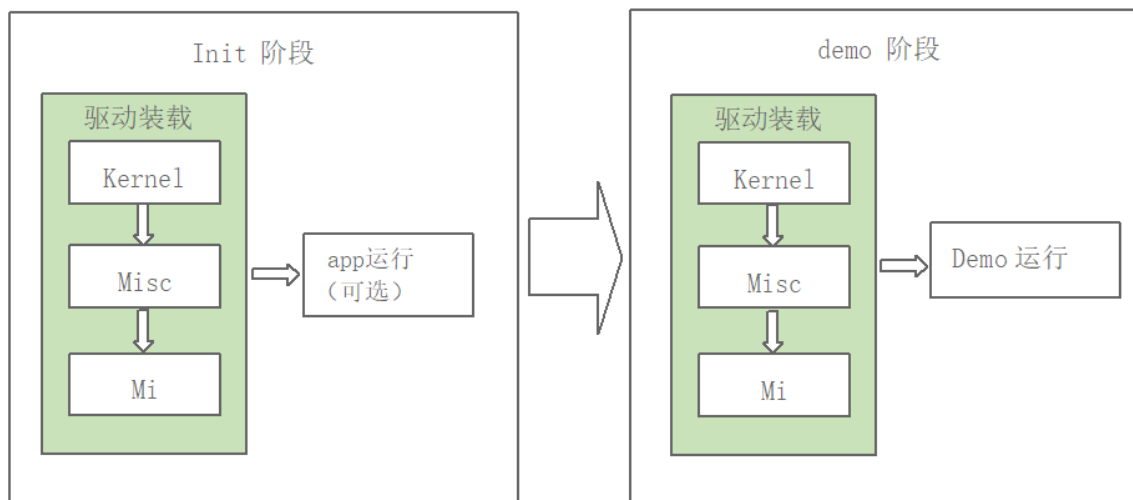
Packaging differences:

- The normal version of lib/ko is packaged in the miservice partition, and then mounted to the /config directory after the kernel is up. The corresponding ko will be insmod after the partition is mounted.
- The FastBoot version lib/ko will be packaged into the rootfs partition according to actual needs, part of which is the fast-start application that needs to be quickly dependent, and part of it can be delayed or packaged

into miservice, where the ko in rootfs will be in init.sh The stage is insmoded, which saves the waiting time for mounting and achieves the acceleration effect.

Let's look at the implementation of init.sh again to understand the following content easily:

后启动流程这个阶段包括分区的挂载与modules的装载，大致分为两个阶段：init 阶段 与 demo 阶段：



init 阶段：（ refer to /etc/init.sh ）

该阶段进行系统必要的环境配置

- 装载系统环境配置必要驱动
依次装载kernel，misc，mi驱动
- app 运行
该demo需在后台运行，存放于dram

demo阶段：（ refer to demo.sh ）

该阶段进行demo运行的其它操作

- 装载次要驱动
依次装载kernel，misc，mi驱动
- demo 运行

该demo 存放于flash

2.3. 模块装载配置

本小节讲解模块的装载相应的配置文件，可以根据需求灵活配置。

Fastboot版本rootfs打包参考：project\image\configs\i2m\rootfs_fastboot.mk 中的做法，将需要提前启动的ko/lib打包到rootfs分区，然后将对应insmod ko/启动app的操作集成到init.sh文件即可

如下，将.mod_depend中指定的mi相关ko的insmod动作放到init.sh去做：

```
129  #-----mi ko start.-----#
130  if [ -f "$($RELEASE_ROOT)/modules/$($KERNEL_VERSION)/.mods_depend" ]; then \
131  cat $($RELEASE_ROOT)/modules/$($KERNEL_VERSION)/.mods_depend | \
132  sed '2,20s#\(.*)#insmod /lib/modules/$($KERNEL_VERSION)/\1.ko\nif [ $$? -eq 0 ]; then\n busybox mknod
133  cat $($LIB_DIR_PATH)/modules/$($KERNEL_VERSION)/.mods_depend | \
134  sed 's#\(.*)#$(LIB_DIR_PATH)/modules/$($KERNEL_VERSION)/\1.ko#' | \
135  xargs -i cp -rvf {} $($OUTPUTDIR)/rootfs/lib/modules/$($KERNEL_VERSION); \
136  echo "#mi module" >> $($INIT_FILE); \
137  fi;
138  # misc late
139  if [ -f "$($RELEASE_ROOT)/modules/$($KERNEL_VERSION)/misc_mod_list_late" ]; then \
140  cat $($RELEASE_ROOT)/modules/$($KERNEL_VERSION)/misc_mod_list_late | \
141  sed 's#\(.*)#insmod /lib/modules/$($KERNEL_VERSION)/\1.ko#' >> $($INIT_FILE); \
142  cat $($RELEASE_ROOT)/modules/$($KERNEL_VERSION)/misc_mod_list_late | \
143  sed 's#\(.*)#$(RELEASE_ROOT)/modules/$($KERNEL_VERSION)/\1.ko#' | \
144  xargs -i cp -rvf {} $($OUTPUTDIR)/rootfs/lib/modules/$($KERNEL_VERSION); \
145  echo -e "# misc_mod_list_late\n" >> $($INIT_FILE); \
146  fi;
147  #-----misc ko end.-----#
148
```

3. 客制化说明

3.1. fastboot内存统计

以zk_mini_sercurity_fastboot为例统计公版fastboot内存使用情况。

ramdisk占用内存：

文件名称	创建时间	文件类型	文件大小
customer.ubifs	2020/3/24 18:13	UBIFS 文件	4,836 KB
ipl_cust_s.bin	2020/3/24 18:13	BIN 文件	384 KB
ipl_s.bin	2020/3/24 18:13	BIN 文件	384 KB
kernel	2020/3/24 18:13	文件	2,259 KB
miservice.sqfs	2020/3/24 18:13	SQFS 文件	2,616 KB
rootfs.ramfs	2020/3/24 18:13	RAMFS 文件	6,641 KB
uboot_s.bin	2020/3/24 18:13	BIN 文件	768 KB

对应占用的内存：0x67ffff = 6.66M，实际这里应该对应的是rootfs分区大小。

```
/ # cat /sys/kernel/debug/memblock/reserved
0: 0x20004000..0x20007fff
1: 0x20008240..0x203f864f
2: 0x21000000..0x2167ffff
3: 0x24a00000..0x24bfffff
4: 0x24c09000..0x24c3cfff
5: 0x24c3f00c..0x24cfefff
6: 0x24cff040..0x24cff87b
7: 0x24cff880..0x24cff8bb
8: 0x24cff8c0..0x24cff937
9: 0x24cff940..0x24cff947
10: 0x24cff980..0x24cff987
11: 0x24cff9c0..0x24cff9c3
```

所以linux可用内存 = MemTotal - Ramdisk Memory

注：DDR SIZE = MMA SIZE + MemTotal + kernel txt + MMAP IP

3.2. 主要实现流程

fastboot实现的方式主要体现在：project\image\configs\i2m\rootfs_fastboot.mk

这个mk里面把开机的行为全部放到/etc/profile下面，大概流程如下：

1. 将开机必须用到的模块(ko/bin等资源)放到/etc/init.sh下执行

```
Line 67:  chmod 755 $(LATE_INIT_FILE)
Line 97:  sed 's#\(.*\)#insmod /lib/modules/$(KERNEL_VERSION)/\1.ko#> $(INIT_FILE); \
Line 101:  echo -e "# kernel_mod_list\n" >> $(INIT_FILE); \
Line 109:  sed 's#\(.*\)#insmod /config/modules/$(KERNEL_VERSION)/\1.ko#> $(LATE_INIT_FILE); \
Line 113:  echo -e "# kernel_mod_list\n" >> $(LATE_INIT_FILE); \
Line 120:  sed 's#\(.*\)#insmod /lib/modules/$(KERNEL_VERSION)/\1.ko#> $(INIT_FILE); \
Line 124:  echo -e "# misc_mod_list\n" >> $(INIT_FILE); \
Line 130:  sed '2,20s#\(.*\)#insmod /lib/modules/$(KERNEL_VERSION)/\1.ko\nif [ $$? -eq 0 ]; then\n busybox mknod
Line 134:  echo "#mi module" >> $(INIT_FILE); \
Line 139:  sed 's#\(.*\)#insmod /lib/modules/$(KERNEL_VERSION)/\1.ko#> $(INIT_FILE); \
Line 143:  echo -e "# misc_mod_list_late\n" >> $(INIT_FILE); \
Line 155:  sed -i 's/mi_common/insmod /lib/modules/$(KERNEL_VERSION)/mi_common.ko\nmajor=\`cat /proc/devices | busy
Line 156:  echo "#mi module" >> $(INIT_FILE); \
Line 157:  sed -i '/#mi module/a major=\`cat /proc/devices | busybox awk "```$2==`"mi_poll`" {print ```$1}``\nbus
Line 174:  echo mdev -s >> $(INIT_FILE); \
Line 177:  sed -i 's/mi_sys.ko/mi_sys.ko cmdQBufSize=128 logBufSize=0 default_config_path=/sstar_configs/g' $(INIT_FI
Line 178:  sed -i 's/fbdev.ko/fbdev.ko default config path file=/sstar_configs/fbdev.ini/g' $(INIT_FILE); \
```

2. 在init.sh最后启动app，

sdk\verify\application\zk_mini_secrurity_fastboot\image.mk

```

app:
ifeq ($(IMAGE_INSTALL_DIR),)
@echo "directory of image is not defined"
@exit 1
endif
@rm -rf $(IMAGE_INSTALL_DIR)/rootfs/customer_app/lib
@mkdir -p $(IMAGE_INSTALL_DIR)/rootfs/customer_app/lib

#base

@cp -rf $(APPLICATION_PATH)/zk_mini_security_fastboot/RunFiles/bin/zkgui $(IMAGE_INSTALL_DIR)/rootfs/customer_app/zkgui
@chmod 777 $(IMAGE_INSTALL_DIR)/rootfs/customer_app/zkgui
$(TOOLCHAIN_REL)strip --strip-unneeded $(IMAGE_INSTALL_DIR)/rootfs/customer_app/zkgui
@cp -rfd $(APPLICATION_PATH)/zk_mini_security_fastboot/RunFiles/lib/RunFirst/lib $(IMAGE_INSTALL_DIR)/rootfs/customer_app/
@cp -rfd $(APPLICATION_PATH)/zk_mini_security_fastboot/RunFiles/lib/RunFirst/libsecurity $(IMAGE_INSTALL_DIR)/rootfs/customer_app/
$(TOOLCHAIN_REL)strip --strip-unneeded $(IMAGE_INSTALL_DIR)/rootfs/customer_app/lib/*
@cp -rfd $(APPLICATION_PATH)/zk_mini_security_fastboot/RunFiles/lib/RunLater/lib $(IMAGE_INSTALL_DIR)/customer/lib
$(TOOLCHAIN_REL)strip --strip-unneeded $(IMAGE_INSTALL_DIR)/customer/lib/*
@cp -rf $(APPLICATION_PATH)/zk_mini_security_fastboot/RunFiles/res/RunFirst/* $(IMAGE_INSTALL_DIR)/rootfs/customer_app/
@cp -rf $(APPLICATION_PATH)/zk_mini_security_fastboot/RunFiles/res/RunLater $(IMAGE_INSTALL_DIR)/customer/res
@cp -rf $(APPLICATION_PATH)/zk_mini_security_fastboot/RunFiles/etc/* $(IMAGE_INSTALL_DIR)/rootfs/etc/
@cat $(APPLICATION_PATH)/zk_mini_security_fastboot/RunFiles/demo.sh >> $(IMAGE_INSTALL_DIR)/rootfs/etc/init.sh
@cp $(APPLICATION_PATH)/zk_mini_security_fastboot/RunFiles/run_later.sh $(IMAGE_INSTALL_DIR)/customer/
@chmod 777 $(IMAGE_INSTALL_DIR)/customer/run_later.sh
@echo "nameserver 202.96.128.86" >> $(IMAGE_INSTALL_DIR)/rootfs/etc/resolv.conf
@echo "nameserver 8.8.8.8" >> $(IMAGE_INSTALL_DIR)/rootfs/etc/resolv.conf

```

```

/ # cat /etc/init.sh
insmod /lib/modules/4.9.84/mhal.ko
#misc_mod_list
insmod /lib/modules/4.9.84/mi_common.ko
insmod /lib/modules/4.9.84/mi_sys.ko cmdQBufSize=768 logBufSize=256 config_ini_path="/sstar_co
_config"
insmod /lib/modules/4.9.84/mi_gfx.ko
insmod /lib/modules/4.9.84/mi_panel.ko
insmod /lib/modules/4.9.84/mi_disp.ko
#mi module 1
insmod /lib/modules/4.9.84/fbdev.ko default_config_path_file=/sstar_config/fbdev.ini

#misc_mod_list_late
#kernel_mod_list
mdev -s
date 032912122020.12
mkdir -var/tmp
export LD_LIBRARY_PATH=$LD_LIBRARY_PATH:/usr/local/lib:/customer_app/lib:/customer_app/libsecu
cd /customer_app
chmod 777 zkgui
./zkgui &

```

注：app启动显示第一张画面必须的res放在rootfs的customer_app中，其他的res放在customer下，这样做的好处是可以减少rootfs的大小，加快启动速度。

3. app起来后，再去mount非必要的分区和挂载

```

and ramfs=squashfs.p2.partition.config3 profile infinity2m_ssc01a_s01a_fastboot_defconfigX
#!/bin/sh
export PATH=/bin:/sbin:/usr/bin:/usr/sbin
export LD_LIBRARY_PATH=/lib
mkdir -p /dev/pts
ulimit -c unlimited
export PATH=$PATH:/config
export TERMINFO=/config/terminfo
mkdir -p /dev/pts
mount -t tmpfs mdev /dev
mdev -s
mkdir -p /dev/pts
if [ -e /etc/init.sh ]; then
.../etc/init.sh
fi
mount -t squashfs /dev/mtdblock3 /config
mount -t squashfs /dev/mtdblock4 /customer
mount -t jffs2 mtd:appconfigs /appconfigs

export LD_LIBRARY_PATH=$LD_LIBRARY_PATH:/customer/lib:/config/lib
busybox telnetd&
sleep 8
/customer/demo.sh

```

3.3. 软件客制化内容

客户在导入fastboot时，主要需要考虑以下几个地方。

3.3.1 客户的app大小跟公版不同

app的大小对fastboot至关重要，因为app越大，制作出来的ramdisk越大，rootfs的分区越大，使用的内存也越大，同时解压ramdisk花的时间也就越长。所以如果客户的app很大，fastboot并不能发挥它的优势。

目前app是放在rootfs下面，所以如果客户的app大小跟公版不同，可以通过修改如下分区文件来更改rootfs分区的大小：

Nand：project\image\configs\i2m\spinand.ramfs-squashfs.p2.partition.config

```
rootfs$(RESOUC) ... = $(OUTPUTDIR)/rootfs
rootfs$(FSTYPE) ... = ramfs
rootfs$(PATSIZE) ... = 0x00680000
rootfs$(BOOTENV) ... = console=ttyS0,115200 ubi.mtd=UBI,2048 rootfstype=ramfs initrd=
rootfs$(BOOTCMD) ... = nand read.e $(INITRAMFSLOADADDR) rootfs $(rootfs$(PATSIZE))\;
rootfs$(MTDPART) ... = $(rootfs$(PATSIZE)) (rootfs)
```

Nor：project\image\configs\i2m\nor.ramfs.partition.config

```
kernel$(RESOUC) ... = $(PROJ_ROOT)/release/$(PRODUCT)/$(CHIP)/$(BOARD)/$(TOOLCHAIN)/$
kernel$(PATSIZE) ... = 0x00210000
kernel$(BOOTENV) ... = $(KERNEL_BOOT_ENV) loglevel=0
kernel$(BOOTCMD) ... = sf read $(KERNELBOOTADDR) \${sf_kernel_start} \${sf_kernel_si

rootfs$(RESOUC) ... = $(OUTPUTDIR)/rootfs
rootfs$(FSTYPE) ... = ramfs
rootfs$(PATSIZE) ... = 0x00680000
rootfs$(BOOTENV) ... = rootfstype=ramfs initrd=$(INITRAMFSLOADADDR),$(rootfs$(PATSIZE))
rootfs$(BOOTCMD) ... = mxc r.info rootfs\; sf read $(INITRAMFSLOADADDR) \${sf_part_st
```

注：分区总大小不能超过flash的总大小，所以rootfs增大后，需要从其他分区相应的减少；同时分区大小必须nor 64kb对齐，nand 128k对齐。

3.3.2 客户的app内存使用跟公版不同

目前我们DDR的内存主要由mma和linux memory组成：

DDR SIZE ~= MMA SIZE + linux MemTotal

所以客户可以根据实际mma的使用情况，增大或减少mma的大小来调整内存使用，实际mma的使用可以在最大使用场景通过如下命令dump出来：

```
/ # cat /proc/mi_modules/mi_sys_mma/mma_heap_name0
mma_heap_name heap_base_cpu bus_addr length chunk_mgr_avail
mma_heap_name0 2299a000 1566000 a3c000

chunk_mgr info:
offset length avail
0 1566000 a3c000

each chunk info:
offset length used_flag task name
0 20000 1 CMDMEM
20000 1000 1 sys_logConfig
21000 40000 1 fb_device
421000 177000 1 gui
598000 4000 1 gui_readbmp1
59c000 4000 1 gui_readbmp1
5a0000 258000 1 gui_readbmp1
7f8000 10000 1 gui_readbmp1
808000 10000 1 gui_readbmp1
818000 10000 1 gui_readbmp1
828000 10000 1 gui_readbmp1
838000 10000 1 gui_readbmp1
848000 10000 1 gui_readbmp1
858000 10000 1 gui_readbmp1
```

3.3.3 客户的app行为跟公版不同

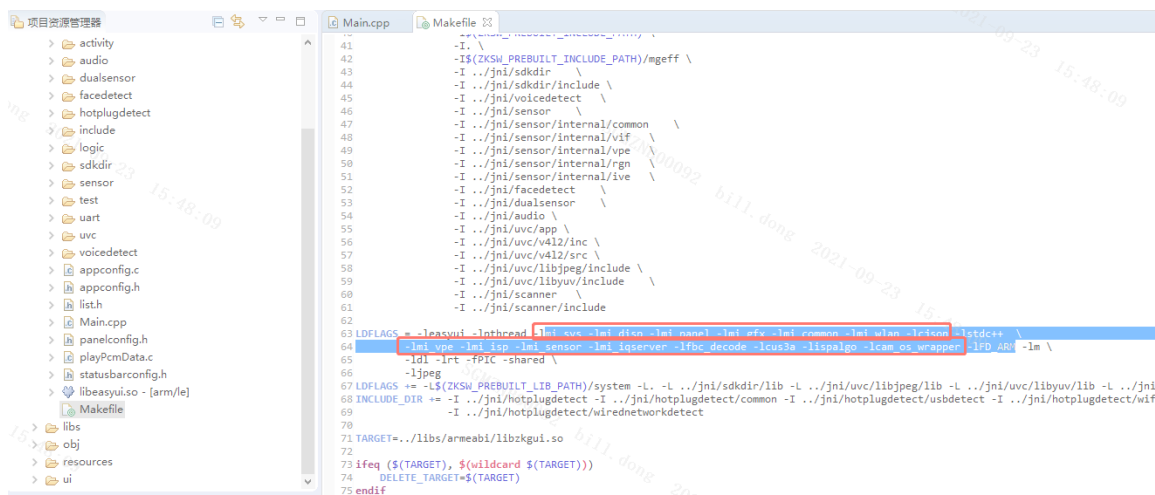
fastboot跟正常启动最大的区别在于fastboot有些非必要的模块是延迟启动的，所以画面刚出来不能马上使用usb或者wifi等没有启动的模块，需要在app的ui做等待处理。

3.3.4 dlopen的处理方式

编译app时只保留必须的动态库，其他动态库使用dlopen的方式加载：减小rootfs的大小。这就要求做好规划哪些放到init.sh，哪些放到demo.sh。

公版举例Zk Demo如下：

Makefile如下内容是必须保留的动态库，其他使用dlopen的方式加载。



dlopen的使用示例:

```
22 static RTSPClientAssembly_t g_stRTSPClientAssembly;
23
24 static int OpenRTSPLibrary()
25 {
26     g_stRTSPClientAssembly.pHandle = dlopen("librtsp.so", RTLD_NOW);
27     if(NULL == g_stRTSPClientAssembly.pHandle)
28     {
29         printf("%s: Can not load librtsp.so!, dLError: %s\n", __func__, dLError());
30         return -1;
31     }
32
33     g_stRTSPClientAssembly.pfnrtsp_init = (int (*)(const char *liv555Url, const char *configIni, unsigned int width,
34     if(NULL == g_stRTSPClientAssembly.pfnrtsp_init)
35     {
36         printf("%s: dlsym rtsp_init failed, %s\n", __func__, dLError());
37         return -1;
38     }
39
40     g_stRTSPClientAssembly.pfnrtsp_deinit = (void (*)(void))dlsym(g_stRTSPClientAssembly.pHandle, "rtsp_deinit");
41     if(NULL == g_stRTSPClientAssembly.pfnrtsp_deinit)
42     {
43         printf("%s: dlsym rtsp_deinit failed, %s\n", __func__, dLError());
44         return -1;
45     }
46
47     return 0;
48 }
```

3.4. Flash驱动客制化内容

公版flash驱动默认没有跑到104M，如果公版默认软件（54M）不能满足开机时间，需要专门调整对应flash驱动默认的速度。同时需要按照如下硬件内容PCB需要找原厂专门check。

3.5. 客制化内容

1. Fastboot开机速度提高，需要flash跑到104M，才能开机更快。这就需要FLASH相关的PCB需要找原厂专门check。

2. 不同Flash厂家的速度差异比较大。
3. Fastboot如需打开bootlogo，开bootlogo必须要运行uboot，具体改动如下：
- a · setenv ota_upgrade_status 1，通过ota_upgrade_status控制是否运行uboot
 - b · setenv autoestart 0，关闭uboot网络功能，加快开机时间
 - c · 打开Flash Quad Read Mode: CONFIG_MS_SPINAND_QUAD_READ (需要flash支持)
-

4. 开机时间统计

启动阶段	耗时(ms)
IPL~UBOOT IPL gbf78c2e - runUBOOT()	65
UBOOT~KERNEL runUBOOT() - Starting kernel	1168
KERNEL~KO INSMOD FINISH Starting kernel - mknod: /dev/mi_poll: File exists	2061
INSMOD FINISH - APP START DONE mknod: /dev/mi_poll: File exists - FB_AllocHWSurface 1303	667
Total :	3961

test environment:

- flash model: MX35LF1GE4AB[128M](C2-12)
- Software version: TAKOYAKI_DLS00V008

Test Conditions:

- Open the boot logo: setenv ota_upgrade_status 1

- uboot open quadmode: `#define SUPPORT_SPINAND_QUAD (1)`
- Turn off uart eth: `setenv autoestart 0`